

PROJECT DOCUMENTATION

Coffee Loyalty Platform

NFC-based “buy N, get 1 free” loyalty system for coffee vendors — mobile app, vendor dashboard and admin dashboard.

Version 1.2 · 26 September 2026

Contents

1. Introduction

1.1 Overview

1.2 Actors and roles

1.3 Core concepts and business rules

2. Diagrams

2.1 Sequence diagram: earning a point

2.2 Sequence diagram: redeeming a free coffee

2.3 Entity Relationship Diagram

3. Flows and Example

3.1 Scan flow (earning a point)

3.2 Redemption flow

3.3 Worked example: Joy Corner

4. Database

4.1 Table overview

4.2 Table definitions

4.3 Indexes worth adding on day one

5. Technical Architecture

5.1 Tech stack

5.2 Supporting services

5.3 Backend implementation decisions

5.4 Build order

5.5 NFC hardware and security

6. API Endpoints

6.1 Endpoints by client

6.2 Mobile app (/api/app) — customer token

6.3 Vendor dashboard (/api/vendor) — staff token, always filtered to the staff's vendor

6.4 Admin dashboard (/api/admin) — platform admin token

7. Dashboards

7.1 Vendor vs admin dashboard

8. Edge Cases and Fraud

8.1 Edge cases and failures

8.2 Automatic fraud flags

9. Decisions and Roadmap

9.1 Decisions

9.2 Later versions

What changed in v1.2: document reorganised into sections; customer login moved to Google, Facebook and Apple sign-in (new `user_identities` table); new `fraud_flags` table; NTAG 424 DNA confirmed from day one; new columns on `platform_admins`, `scan_sessions` and `point_events`; new redemption sequence diagram and Joy Corner worked example. Items new in this version are marked **NEW**.

SECTION 1

Introduction

What the platform is, who uses it and the rules it follows.

1.1 Overview

A multi-vendor loyalty platform for coffee shops (drive-thru, takeaway): buy a set number of coffees at a vendor (default 5, set by each vendor), get 1 free. Each purchase earns a point when shop staff tap the branch's NFC card on the customer's phone.

The platform has three parts:

Part	Users	Purpose
Mobile app (iOS + Android)	Customers	Sign in, request a point, see balances and history per vendor, see how many coffees remain to a reward
Vendor dashboard (web)	Vendor owners, branch managers	See their own branches, points issued, redemptions, staff and NFC cards
Admin dashboard (web)	Platform team	Manage all vendors, onboard new ones, issue and revoke NFC cards, monitor usage and fraud

The free coffee is paid for by the vendor. For the vendor, the platform is a customer-retention and advertising channel.

v1 scope: earn points by NFC tap, redeem at the store, per-vendor balances, both dashboards. Point expiry and push notifications are deferred (see Roadmap).

1.2 Actors and roles

Four roles, each scoped to what they own.

Role	Scope	Can do
Customer	Own account	Sign in, request points, view wallets and history, redeem rewards
Staff	One branch	Tap the branch NFC card to award or redeem; optionally view today's activity
Vendor admin	One vendor (all its branches)	Manage branches, staff, loyalty program settings; view reports for their vendor only

Role	Scope	Can do
Platform admin	Everything	Onboard vendors, issue/revoke NFC cards, view all vendors, investigate fraud, support users

Staff and vendor admins log in to the vendor dashboard; platform admins log in to the admin dashboard. Customers only use the mobile app.

1.3 Core concepts and business rules

Points are earned at a branch but stored in a wallet at the vendor level.

- **Vendor vs branch.** A vendor is the brand (e.g. a coffee chain). A branch is one physical shop. A chain with 6 drive-thrus = 1 vendor, 6 branches.
- **Wallet.** One wallet per customer per vendor. Points from any branch of the same vendor add to the same wallet. Points never move between vendors.
- **Program.** Each vendor has a loyalty program (its “settings”): points required (default 5) and the reward (default 1 free coffee). Programs are versioned: if a vendor changes 5 to 8, existing wallets keep their original program until the reward is claimed.
- **Earning.** 1 tap = 1 point. The server awards the point, never the app. Each award is recorded as a row in the point ledger and the wallet balance increases by 1 in the same transaction.
- **Rate limit.** Maximum 1 point per customer per branch every 10 minutes (configurable per vendor).
- **Redemption.** When balance reaches the program’s target, the customer can redeem at any branch of that vendor. The customer chooses when to claim; staff tap the card in redeem mode; the server deducts the points and records the redemption. The vendor pays for the free coffee.
- **Ledger is the source of truth.** The wallet balance is a cached total. It must always equal the sum of that wallet’s point events (earns are +1, redemptions are negative).

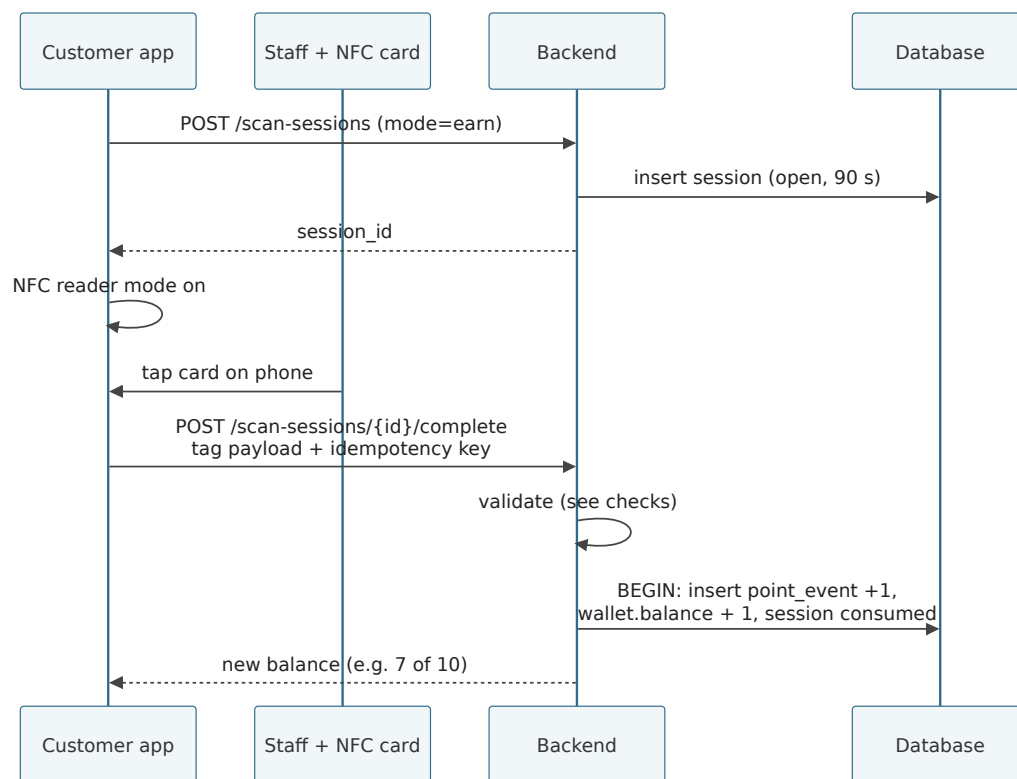
SECTION 2

Diagrams

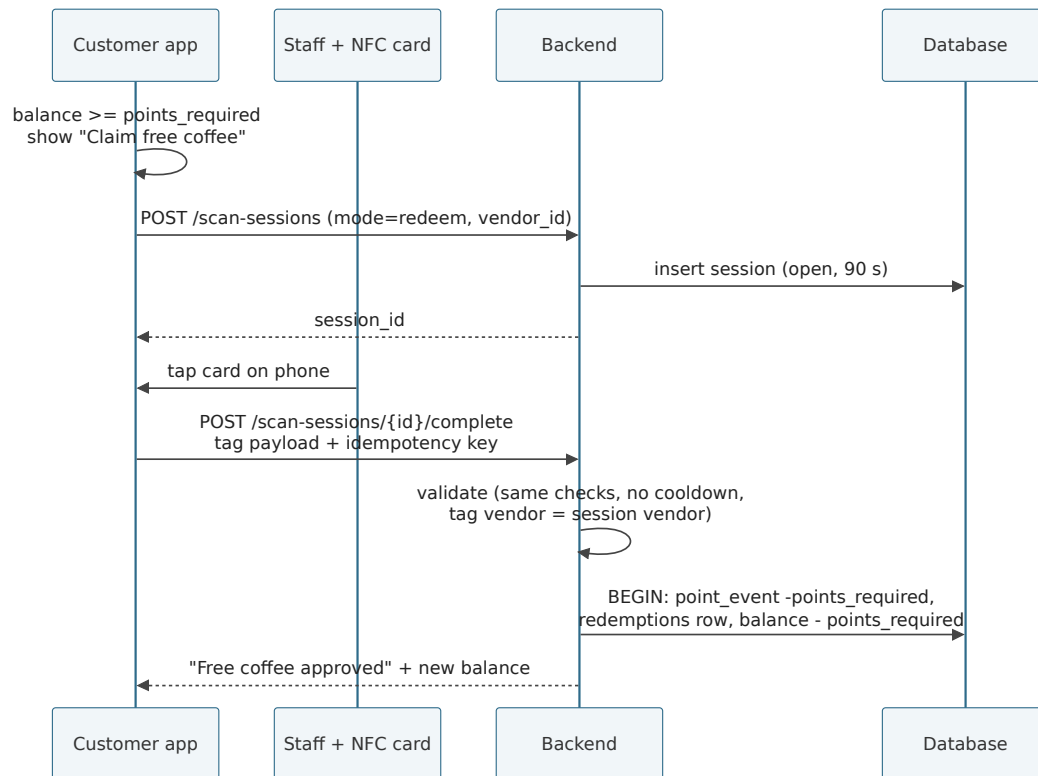
Entity relationship diagram and the two main sequence diagrams.

2.1 Sequence diagram: earning a point

All diagrams in one place. The ERD shows how the tables connect; the two sequence diagrams show who talks to whom when a point is earned and when a free coffee is redeemed. Section 3 explains each flow step by step.

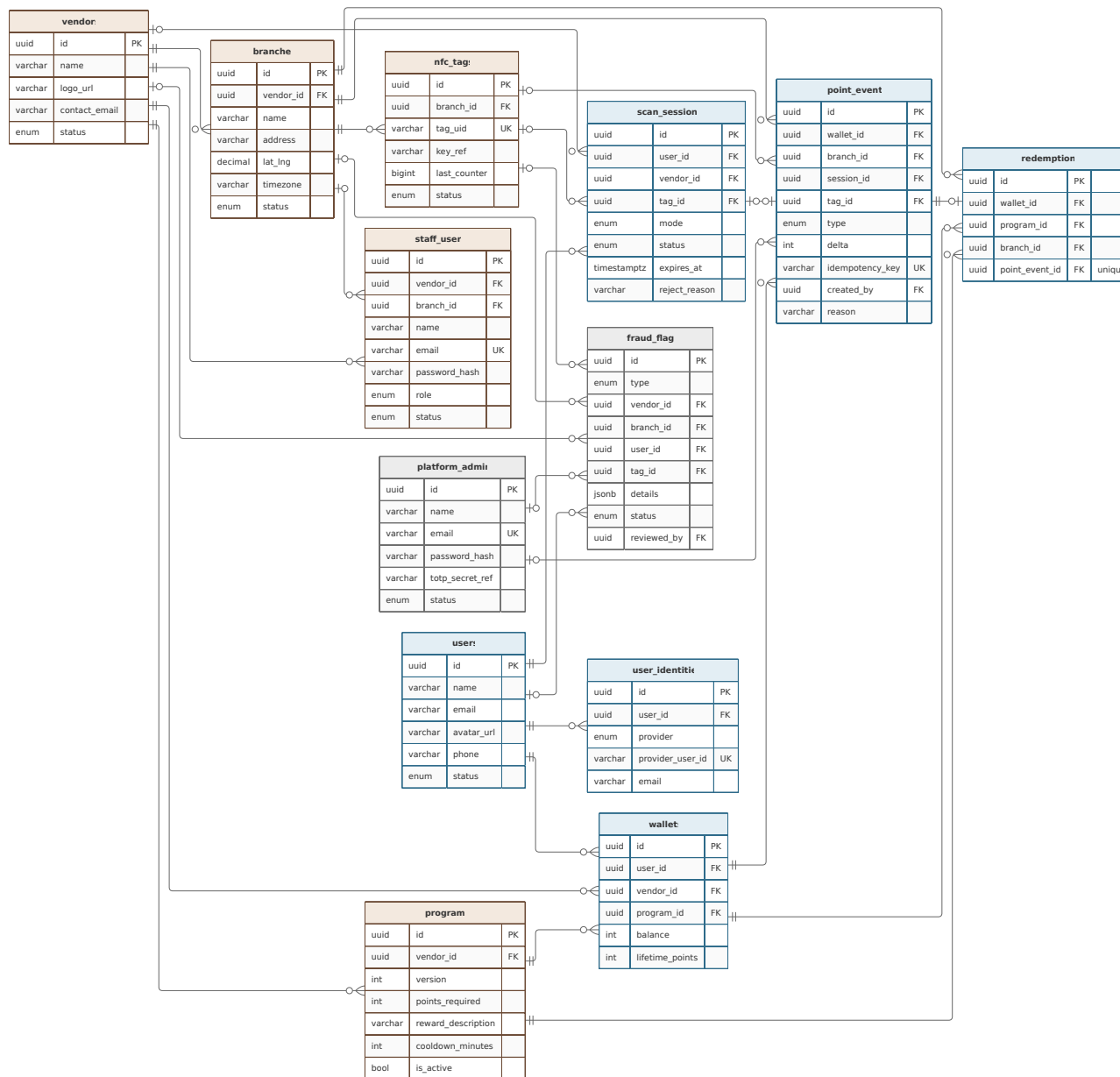


2.2 Sequence diagram: redeeming a free coffee



2.3 Entity Relationship Diagram

Thirteen tables; **vendors** is the tenant root and almost every row traces back to one vendor. Brown = vendor side, blue = customer and activity side, grey = platform. Lines connect the foreign key to the primary key it references (|| = exactly one, O< = zero or many, O| = zero or one). New in v1.2: **user_identities**, **fraud_flags**.



Flows and Example

How earning and redeeming work, then a full walk-through with a real vendor.

3.1 Scan flow (earning a point)

The app opens a session, reads the tag, and the server decides; the balance updates within about 2 seconds of the tap.

Sequence diagram: see Diagram 2.2.

1. Customer taps **Get my point** in the app.
2. App calls `POST /scan-sessions` with `mode=earn`. Server creates an open session that expires in 90 seconds.
3. App switches the phone to NFC reader mode and shows “Hand your phone to the barista”.
4. Staff tap the branch card on the phone. The app reads the tag payload.
5. App sends the payload, the session id and a fresh idempotency key to `POST /scan-sessions/{id}/complete`.
6. Server runs the checks below. If all pass, in one database transaction: insert a `point_events` row (+1), increment `wallets.balance` (creating the wallet on first visit), mark the session consumed, update `nfc_tags.last_counter`.
7. App shows the new balance and how many coffees remain.

Server checks, in order

1. Session exists, belongs to this user, is open and not expired.
2. Idempotency key already used → return the original result, award nothing new.
3. Tag exists and is `active`; resolve its branch and vendor.
4. Tag signature valid and counter greater than `last_counter` (NTAG 424 DNA).
5. User not blocked; vendor and branch active.
6. Cooldown: no earn event for this wallet at this branch within `cooldown_minutes`.

Any failure marks the session `rejected` with a `reject_reason` and returns a clear message to the app.

3.2 Redemption flow

Sequence diagram: see Diagram 2.3.

Redemption reuses the scan mechanism with `mode=redeem`, so a full card can only be spent once, at one branch.

1. When `balance >= points_required`, the app shows a **Claim free coffee** button for that vendor.
2. Customer taps it; app opens a session with `mode=redeem` and the vendor id (stored in `scan_sessions.vendor_id`).
3. Staff tap the branch card, exactly as when earning.
4. Server runs the same checks (minus cooldown), plus: the tag's vendor matches the session's vendor, and balance is still high enough.
5. In one transaction: insert a `point_events` row with `type=redeem` and `delta = -points_required`, decrement `wallets.balance`, insert a `redemptions` row, and if the vendor has a newer program version, move the wallet onto it.
6. App shows a confirmation screen the staff can see ("Free coffee approved", branch name, time). Staff serve the coffee.

Extra points above the target carry over: a customer at 6 of 5 who redeems is left with 1.

3.3 Worked example: Joy Corner NEW

How one vendor's rule travels through the tables, from setup to a free coffee. Joy Corner is a vendor with a Smouha branch; its rule is "buy 10 coffees, get 1 free".

Setup (done once)

Table	Row written
<code>vendors</code>	Joy Corner, status = active
<code>branches</code>	Joy Corner Smouha → vendor Joy Corner
<code>nfc_tags</code>	One NTAG 424 DNA card → branch Smouha, with its <code>key_ref</code>
<code>programs</code>	vendor Joy Corner, version 1, points_required = 10 , reward "1 free coffee", cooldown 10 min, <code>is_active = true</code>
<code>users</code> + <code>user_identities</code>	Karim signs in with Google: one user row, one identity row (provider = google)

Step by step

1. Joy Corner has a `programs` row with `points_required = 10`.
2. Customer presses **Request a point** → a `scan_sessions` row is created (mode = earn).
3. Staff taps the card → the API finds the tag → branch → vendor, and checks the cooldown and the tag counter.
4. In one transaction: insert `point_events` (+1), `wallets.balance` + 1, session consumed. On the first visit the wallet is created here and linked to program version 1.
5. At balance 6 nothing special happens: it is an ordinary paid coffee. At 10 the app shows **Claim free coffee**.

6. Customer claims → redeem session (vendor = Joy Corner) → staff taps → `point_events` (−10) + `redemptions` row + `balance` − 10.

Karim's wallet over time

Visit	What happens	Rows written	Balance	App shows
1	Buys a coffee, staff tap	wallet created; <code>point_events</code> +1	1	1 of 10 · 9 to go
2-5	Buys a coffee each visit	<code>point_events</code> +1 each	5	5 of 10
5 (again)	Taps twice within 10 min	Session rejected, reason = cooldown	5	"Already earned here, try again in X min"
6	Buys a coffee (ordinary, paid)	<code>point_events</code> +1	6	6 of 10 · 4 to go
7-10	Buys a coffee each visit	<code>point_events</code> +1 each	10	10 of 10 · Claim free coffee
11	Buys, staff tap first (earn)	<code>point_events</code> +1	11	Claim free coffee
11	Claims the reward, staff tap (redeem)	<code>point_events</code> −10; <code>redemptions</code> row	1	"Free coffee approved" · 1 of 10

The extra point carries over ($11 - 10 = 1$). At any time, `wallets.balance` equals the sum of Karim's `point_events.delta` for Joy Corner; `lifetime_points` keeps counting up (11) and never decreases.

If Joy Corner changes the rule. If Joy Corner publishes version 2 ("buy 8, get 1 free"), a new `programs` row is added and becomes the active one. Karim's wallet stays on version 1 (10 points) until he redeems; the redeem transaction then moves his wallet to version 2.

SECTION 4

Database

All thirteen tables, their columns, constraints and indexes.

4.1 Table overview

#	Table	Side	Holds
1	vendors	Vendor	Coffee brands, e.g. Joy Corner
2	branches	Vendor	Physical shops of a vendor
3	staff_users	Vendor	Vendor owners, branch managers, baristas
4	nfc_tags	Vendor	The NTAG 424 DNA card at each branch
5	programs	Vendor	Each vendor's reward rules ("settings"), versioned
6	users	Customer	Customers of the mobile app
7	user_identities NEW	Customer	Google / Facebook / Apple accounts linked to a customer
8	wallets	Customer	One balance per customer per vendor
9	scan_sessions	Activity	Each "request a point" or "claim" press, valid 90 s
10	point_events	Activity	The ledger: every +1 and every -N, append-only
11	redemptions	Activity	Every free coffee given
12	platform_admins	Platform	Your team (admin dashboard)
13	fraud_flags NEW	Platform	Suspicious activity for admins to review

Every table has `created_at` and `updated_at` (timestamps, UTC) in addition to the columns listed in 4.2. Soft delete via `status` rather than removing rows.

4.2 Table definitions

vendors

Column	Type	Notes
id	uuid PK	
name	varchar(120)	Brand name shown in the app
logo_url	varchar	
status	enum	active, suspended
contact_email	varchar	

branches

Column	Type	Notes
id	uuid PK	
vendor_id	uuid FK → vendors	
name	varchar(120)	e.g. “Smouha drive-thru”
address	varchar	
lat, lng	decimal(9,6)	For map and optional geofence check
timezone	varchar	e.g. Africa/Cairo
status	enum	active, closed

nfc_tags

Column	Type	Notes
id	uuid PK	
branch_id	uuid FK → branches	Which shop holds this card
tag_uid	varchar UNIQUE	Hardware UID of the tag
key_ref	varchar NOT NULL	Reference to the tag’s secret key in a secrets store. Required: NTAG 424 DNA from day one. Never store the raw key here
last_counter	bigint, default 0	Last accepted tap counter; new taps must be higher
status	enum	active, lost, revoked

staff_users

Column	Type	Notes
id	uuid PK	
vendor_id	uuid FK → vendors	Always set
branch_id	uuid FK → branches, nullable	Null for vendor-wide admins
name, email	varchar	email UNIQUE
password_hash	varchar	
role	enum	vendor_admin, branch_manager, staff
status	enum	active, disabled

platform_admins

Column	Type	Notes
id	uuid PK	
name, email	varchar	email UNIQUE
password_hash	varchar	
totp_secret_ref NEW	varchar, nullable	Reference to the 2FA (authenticator app) secret in the secrets store
status NEW	enum	active, disabled

users (customers)

Column	Type	Notes
id	uuid PK	
name	varchar	From the sign-in provider; editable
email	varchar, nullable	Apple may give a private relay address or none
avatar_url NEW	varchar, nullable	Profile picture from the provider
phone	varchar, nullable	Optional; no longer the login (was UNIQUE, OTP in v1.1)
status	enum	active, blocked

user_identities NEW

One row per social account linked to a customer, so the same person can sign in with Google and Apple and reach the same wallets.

Column	Type	Notes
id	uuid PK	
user_id	uuid FK → users	
provider	enum	google, facebook, apple
provider_user_id	varchar	The stable account ID from the provider (the “sub” claim)
email	varchar, nullable	Email the provider returned at sign-in

Constraint: UNIQUE (provider, provider_user_id).

programs

Column	Type	Notes
id	uuid PK	
vendor_id	uuid FK → vendors	
version	int	Increments each time the vendor changes the rules
points_required	int	Default 5 (e.g. 10 for Joy Corner)
reward_description	varchar	e.g. “1 free coffee, any size”
cooldown_minutes	int	Default 10
is_active	bool	Only one active program per vendor

Constraints: UNIQUE (vendor_id, version); partial UNIQUE (vendor_id) WHERE is_active.

wallets

Column	Type	Notes
id	uuid PK	
user_id	uuid FK → users	
vendor_id	uuid FK → vendors	
program_id	uuid FK → programs	The program version this wallet runs under
balance	int	Cached sum of point_events.delta
lifetime_points	int	For stats; never decreases

Constraints: UNIQUE (user_id, vendor_id); CHECK (balance >= 0).

scan_sessions

Column	Type	Notes
id	uuid PK	
user_id	uuid FK → users	
vendor_id NEW	uuid FK → vendors, nullable	Set for redeem sessions: which vendor's wallet to spend from
mode	enum	earn, redeem
tag_id	uuid FK → nfc_tags, nullable	Filled when the tap arrives
status	enum	open, consumed, expired, rejected
expires_at	timestamp	created_at + 90 s
reject_reason	varchar, nullable	e.g. cooldown, tag_revoked, bad_signature

point_events (ledger, append-only)

Column	Type	Notes
id	uuid PK	
wallet_id	uuid FK → wallets	
branch_id	uuid FK → branches	Where it happened
session_id	uuid FK → scan_sessions, nullable	Null for manual adjustments
tag_id	uuid FK → nfc_tags, nullable	
type	enum	earn, redeem, adjust
delta	int	+1 for earn, −points_required for redeem
idempotency_key	varchar UNIQUE	Sent by the app per tap
created_by	uuid FK → platform_admins, nullable	Admin id for adjustments (now a real foreign key)
reason NEW	varchar, nullable	Required for type = adjust

redemptions

Column	Type	Notes
id	uuid PK	
wallet_id	uuid FK → wallets	
program_id	uuid FK → programs	Which rules were redeemed
branch_id	uuid FK → branches	Where the free coffee was served
point_event_id	uuid FK → point_events, UNIQUE	The matching negative ledger row

fraud_flags NEW

A list of suspicious activity for the admin dashboard (see Section 13). The system writes a row when something looks wrong; an admin reviews it. Flags do not block anything on their own.

Column	Type	Notes
id	uuid PK	
type	enum	too_many_earns, branch_spike, tag_counter_anomaly, far_location
vendor_id	uuid FK → vendors, nullable	Who the flag is about (one or more of these is set)
branch_id	uuid FK → branches, nullable	
user_id	uuid FK → users, nullable	
tag_id	uuid FK → nfc_tags, nullable	
details	jsonb	The numbers behind the flag, e.g. {"earns_today": 5}
status	enum	open, dismissed, confirmed
reviewed_by	uuid FK → platform_admins, nullable	Set when an admin closes the flag

4.3 Indexes worth adding on day one

- `point_events (wallet_id, created_at)` for history screens
- `point_events (branch_id, created_at)` for dashboard reports
- `point_events (wallet_id, branch_id, created_at DESC)` for the cooldown check
- `wallets (vendor_id)` for vendor reports
- `user_identities (user_id)` for account lookups NEW
- `fraud_flags (status, created_at)` for the open-flags list NEW

Technical Architecture

Stack, services, implementation decisions and NFC security.

5.1 Tech stack

Decided stack: NestJS (TypeScript) backend, Prisma ORM, PostgreSQL, React web dashboards, Flutter mobile app.

Layer	Choice	Why for this project
Database	PostgreSQL	Relational data and a money-like ledger need ACID transactions; partial unique indexes enforce one active program per vendor; permissions can make <code>point_events</code> append-only
ORM	Prisma	Readable schema file, generated TypeScript types, clean migrations; interactive transactions cover the earn and redeem flows
Backend	NestJS (TypeScript)	One module per domain area, dependency injection and guards for role scoping; TypeScript shared with the web dashboard
Web dashboards	React + TypeScript (Vite)	One app with role-based routes for vendor and admin; no SEO or server rendering needed behind a login. Ant Design, TanStack Query, Recharts
Mobile app	Flutter (Dart)	Close to C# for a .NET / game-dev team; <code>nfc_manager</code> reads NFC tags on iOS and Android; own rendering suits custom reward animations
API contract	OpenAPI (from NestJS)	Generate a typed Dart client for Flutter and a TypeScript client for React from one spec

5.2 Supporting services

- **Customer login:** Google, Facebook and Apple sign-in NEW. The app gets an ID token from the provider; the backend verifies it with that provider, finds or creates the user through `user_identities`, and returns its own access and refresh tokens. Apple sign-in is required by the App Store when other social logins are offered. No SMS/OTP provider is needed.
- **Push notifications (later):** Firebase Cloud Messaging for iOS and Android.

- **Tag secret keys:** cloud secrets manager or KMS, never the database.
- **Hosting:** one backend container plus one managed PostgreSQL instance. No microservices, message queues or Redis in v1.

5.3 Backend implementation decisions

- **Balance changes are atomic conditional updates**, run inside `prisma.$transaction` together with the ledger insert. No row returned means not enough points; two simultaneous redemptions cannot both succeed. Example for redemption:

```
UPDATE wallets
SET balance = balance - 5
WHERE id = $1 AND balance >= 5
RETURNING balance;
```

- **Tenant scoping in NestJS for v1:** a guard reads `vendor_id` (and `branch_id`) from the staff token; every vendor-side service method requires `vendorId`. PostgreSQL row-level security can be added later as a second layer.
- **NTAG 424 verification:** Node's `crypto` has AES but no built-in CMAC; use a well-tested CMAC package or implement RFC 4493 on AES-CBC. Test against the sample vectors in NXP application note AN12196.
- **Fraud checks** NEW: a scheduled job (and a few inline checks in the scan flow) writes suspicious activity to `fraud_flags` for the admin to review. Flags never block a tap by themselves.
- **Modules:** `auth`, `users`, `vendors`, `branches`, `staff`, `nfc-tags`, `programs`, `wallets`, `scan-sessions`, `redemptions`, `fraud-flags`, `admin`, `reports`. `scan-sessions` is the core and is built first, with tests.

5.4 Build order

1. Database schema from the Data model section, migrations, seed data (one vendor, two branches, a few tags)
2. Customer auth (Google, Facebook and Apple sign-in)
3. Scan sessions, earn flow and wallets, tested with NTAG 424 DNA tags
4. Redemption flow
5. Vendor and admin APIs, fraud flag job, then the React dashboards

5.5 NFC hardware and security

Decided: NTAG 424 DNA tags from day one, so every tap produces a one-time signed payload that cannot be copied or replayed.

The customer's phone is the reader; the vendor's card is a passive tag with no battery. iPhones read NFC through Core NFC (iPhone 7 and newer, app in the foreground, a system sheet appears). Android reads through reader mode on phones with NFC hardware.

Option	How it works	Replay protection	Cost per tag (approx.)
Plain tag (NTAG213/215)	Returns a fixed UID every time	None: anyone who reads it once can fake it	~\$0.20
NTAG 424 DNA with SUN (chosen)	Each tap returns UID + counter + cryptographic signature (CMAC) made with a secret key	Strong: server rejects old counters and bad signatures	~\$0.60-1.00

Rules regardless of tag type

- The app never decides points; it only forwards what it read.
- Tag secret keys live in a secrets manager, referenced by `key_ref`, never in app code.
- Lost cards are revoked from the admin dashboard immediately; a replacement tag gets a new UID and key.
- API calls require a signed-in user token over HTTPS.

SECTION 6

API Endpoints

Three API groups, one per client.

6.1 Endpoints by client

Three API groups, one per client, each with its own auth and scoping.

6.2 Mobile app (`/api/app`) — customer token

Method	Path	Purpose
POST	/auth/social NEW	Body {provider, id_token}; verify with Google / Facebook / Apple, find or create the user, return tokens
POST	/auth/refresh NEW	Swap a refresh token for a new access token
GET	/me	Profile
PATCH	/me	Update name, email, phone
GET	/wallets	All wallets: vendor, balance, points_required, remaining
GET	/wallets/{id}/events	Point and redemption history for one vendor
GET	/vendors	Vendors and branches (for discovery and map)
POST	/scan-sessions	Open a session, body {mode} (+ vendor_id for redeem)
POST	/scan-sessions/{id}/complete	Send tag payload + idempotency key; returns new balance

Replaces `/auth/otp/request` and `/auth/otp/verify` from v1.1.

6.3 Vendor dashboard (`/api/vendor`) — staff token, always filtered to the staff's vendor

Method	Path	Purpose
GET	/summary?from&to	Points issued, redemptions, unique customers

Method	Path	Purpose
GET	/branches	List own branches
POST / PATCH	/branches	Add or edit a branch
GET	/events?branch_id&from&to	Ledger rows for own branches
GET	/redemptions	Redemptions for own branches
GET / POST / PATCH	/staff	Manage staff accounts
GET / POST	/programs	View current rules, publish a new version
POST	/tags/{id}/report-lost	Flag a card as lost
GET	/fraud/flags NEW	Flags for own branches (read-only)

6.4 Admin dashboard (/api/admin) — platform admin token

Method	Path	Purpose
GET / POST / PATCH	/vendors	Onboard and manage vendors
GET	/vendors/{id}/summary	Any vendor's stats
POST	/tags	Register a new tag to a branch
PATCH	/tags/{id}	Revoke, reassign
GET	/users/{id}	Customer lookup for support
POST	/wallets/{id}/adjust	Manual point correction (writes an adjust ledger row with reason)
GET	/fraud/flags	Suspicious activity list
PATCH	/fraud/flags/{id} NEW	Dismiss or confirm a flag (sets status and reviewed_by)

SECTION 7

Dashboards

What vendors and platform admins can see and do.

7.1 Vendor vs admin dashboard

The vendor dashboard shows one vendor's own data; the admin dashboard shows every vendor and controls the platform.

Feature	Vendor dashboard	Admin dashboard
Scope	One vendor and its branches	All vendors
Points issued per day / branch	Yes, own only	Yes, all, comparable across vendors
Redemptions (free coffees given)	Yes	Yes
Unique and returning customers	Yes, counts only	Yes
Customer personal details	No (names and emails hidden)	Yes, for support
Branch management	Yes	Yes
Staff accounts	Yes	Yes
Program rules (e.g. 10 → free)	Publish new version	View, override
NFC cards	View, report lost	Register, assign, revoke
Manual point adjustment	No	Yes, with reason
Fraud flags	Own branches, view only	All; dismiss or confirm
Vendor onboarding, billing	No	Yes

Tenant scoping

Every vendor-dashboard query must filter by the logged-in staff member's `vendor_id` (and `branch_id` for branch-level roles). Enforce this in one place: middleware that injects the filter, or database row-level security (PostgreSQL RLS). Never rely on each endpoint remembering to add it.

SECTION 8

Edge Cases and Fraud

What goes wrong in real cafés and how the system handles it.

8.1 Edge cases and failures

Most real problems come from bad café wifi and from people trying to earn points without buying.

Situation	What happens	Handling
Point awarded, response lost	App retries	Same idempotency key → same result, no double point
Session expires while customer fumbles	90 s passes	App opens a new session and retries once, silently
Customer taps twice for one coffee	Second request inside cooldown	Rejected with “Already earned here, try again in X min”
Customer buys 2 coffees at once	Cooldown blocks the 2nd point	v1: accept the limit. Later: staff-side quantity option
Card copied	Fake taps from anywhere	NTAG 424 DNA rejects replayed counters and bad signatures; a <code>tag_counter_anomaly</code> flag is written
Staff give points without a sale	Insider abuse	A <code>branch_spike</code> or <code>too_many_earns</code> flag is written; vendor sees per-branch counts
Card lost or stolen	Anyone can tap it	Revoke from dashboard; all later taps rejected; past points kept
Program changed mid-card	Customer had 3 of 5, new rule is 8	Wallet keeps old program until redeemed
Phone has no NFC	Can’t read the card	v1: not supported, tell user. Later: staff-shown QR fallback
Two redemptions at the same moment	Race condition	Row lock on the wallet during the transaction
Same person signs in with Google, then Apple	Could create two accounts	If the verified email matches an existing user, link a new <code>user_identities</code> row to that user; otherwise offer “link account” in settings

8.2 Automatic fraud flags

Each rule below writes one row to `fraud_flags` with its `type` and the numbers in `details`:

- `too_many_earns`: more than 3 earn events for one user in a day across branches of one vendor.
- `far_location`: taps on one tag from locations far from its branch (if phone location is sent).
- `tag_counter_anomaly`: a tag counter jumping backwards or skipping large ranges.
- `branch_spike`: a branch whose daily points spike above 3× its 30-day average.

SECTION 9

Decisions and Roadmap

What is decided, what is open, and what comes later.

9.1 Decisions

- **Decided:** NTAG 424 DNA tags from day one.
- **Decided:** Customer login with Google, Facebook and Apple sign-in (no phone OTP).
- **Decided:** Separate login tables for customers, vendor staff and platform admins.
- **Decided:** Customer claims the free coffee with a button; it is not given automatically.
- **Still open:** cooldown length: 10 minutes per branch, or per vendor?

9.2 Later versions

Feature	Why	Needs
Push notification on each point	Instant confirmation even if app is closed	FCM / APNs, device tokens table
Point expiry	Limits vendor liability for old unredeemed cards	<code>expires_at</code> rule on programs; ledger already has dates
Staff confirmation app	Second signature on each tap; supports quantity (2 coffees)	Staff mobile app or tablet
QR fallback	Phones without NFC	Rotating QR shown by staff device
Vendor promotions	Double points days, new-customer bonus	Rules engine on programs
Vendor billing	Monetise the platform	Plans table, invoices